AutoBayes: brms in Python

status, demonstrations and a head-to-head comparison with brms

Prepared for Andrew

15 October 2026

Summary

- **Complete:** 25 of the 26 planned brms features are merged into `main`, each accepted only after reproducing brms's own log density, gradient and predictions on frozen fixtures. The 26th, `s()` splines, waits on a licensing decision. Measured against all of brms 2.23 that is roughly a quarter to a third by count (13 of 49 families), concentrated on the models applied researchers fit most (Section 1).
- **What you can do in Python now:** ten model classes, from correlated-slope logistic GLMMs to ordinal, zero-inflated, distributional, multivariate, non-linear, Gaussian-process, autoregressive, mixture and measurement-error models, each specified as in brms and fitted in 2–8 seconds on a laptop CPU (Section 2).
- **Speed:** on four benchmark models with identical data, priors and sampler settings, AutoBayes (NumPyro) went from formula to posterior 3.6–8.5 times faster than brms (rstan). Most of brms's time is compiling its Stan program (26–37 seconds per model), but sampling alone was also roughly 4–9 times faster here. The two posteriors agree within Monte Carlo error (every parameter within 1.9 combined standard errors), with similar effective sample sizes (Section 3). One run each, on a laptop with background load; read the numbers as "several times faster on these models", not as a general constant.
- **Also better:** where brms's own arithmetic loses digits (complementary log-log gradients, censored and truncated Gaussians, zero-inflated and hurdle densities at extreme linear predictors) AutoBayes computes the exact value, verified against 50-digit references (Section 4).
- **Not better yet:** breadth (brms offers far more), maturity, and validation of sampling for the newest model classes; GPU execution is untested.

1 How much of brms is done

The brms-parity loop works through a planned scope of 26 features in three tiers. A feature is accepted only when the model AutoBayes builds reproduces brms's own Stan data, log density, gradient and predictions on frozen fixtures (98 of them, all regenerating identically from brms 2.23.0 and rstan); nothing in that test samples.

Tier	Features	State
A	independent and correlated varying slopes; `g:h`, `g/h`; interactions; transforms; `offset()`; negative binomial, Gamma, lognormal, Beta; probit and cloglog links; learned Student-t ν; brms's default priors	13 of 13 merged
B	distributional formulas (`sigma ~ z`); `weights()`, `cens()`, `trunc()`; `mo()`; cumulative and categorical families; zero-inflated and hurdle Poisson; `s()` splines	5 of 6 merged; `s()` waits on the licensing decision
C	multivariate (`mvbind`, residual correlation); non-linear formulas; Gaussian processes; `ar()`/`ma()`/`arma()`; finite mixtures; `mi()` and `me()`; `log_lik`, `bayes_R2`, PSIS-LOO	7 of 7 merged

Against everything brms 2.23 offers, the picture is narrower. Each feature is a deliberate slice (75 divergence rows list what is refused, with the reason), and several whole areas are not started:

	brms	AutoBayes	not yet offered (examples)
Response families	49	13	Weibull, exponential, skew-normal, beta-binomial, zero-inflated and hurdle negative binomial, `sratio`/`cratio`/`acat`, multinomial, Dirichlet, von Mises, Wiener, Cox
Formula specials	≈19	8	`s()`, `t2()`, `cs()`, `mm()`, `car()`, `sar()`, `cosy()`
Addition terms	≈12	5	`se()`, `rate()`, `thres()`, `subset()`, `dec()`
Prior families	any Stan density	11	horseshoe, R2D2, lasso
Post-processing	≈25	6	`waic`, `kfold`, `loo_compare`, `hypothesis`, `conditional_effects`, `pp_check`

By count that is roughly a quarter to a third of brms. By the models applied researchers most often fit with brms (generalised linear mixed models with correlated slopes, ordinal and categorical outcomes, zero-inflated counts, distributional, multivariate and non-linear models) it is far more than that; the second statement is my judgement, not a measurement.

2 What you can fit in Python today

Ten models, one per class of brms model now offered, each on simulated data with known parameters and fitted with NumPyro on this laptop's CPU (Apple M1 Pro). The code shown is the whole model specification; the full script is `examples/brms_parity/tour.py`. Every fit uses four chains of 1000 warm-up and 1000 kept draws.

Logistic regression with correlated varying intercepts and slopes

```
ab.bernoulli("y ~ x + (1 + x | g)", df, ab.Priors(intercept=ab.Normal(0, 2.5), b=ab.Normal(0, 1),
    sd={"g": {"Intercept": ab.HalfNormal(1), "x": ab.HalfNormal(1)}}, cor={"g": ab.LKJ(2)}))
```

Fit 6.0 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 870. PSIS-LOO $\widehat{\mathrm{elpd}}_{\mathrm{loo}} = -843.4$; Bayes $R^2 = 0.215$.

parameter	true	posterior mean	90% interval
x	0.90	0.76	[0.58, 0.95]
Intercept	-0.40	-0.66	[-0.93, -0.39]
x|g_sigma	0.40	0.53	[0.35, 0.72]
1|g_sigma	0.80	0.92	[0.72, 1.16]
cor|g	0.00	-0.15	[-0.50, 0.21]

Ordinal (cumulative logit) regression with a varying intercept

```
ab.cumulative("y ~ x + (1 | g)", df, ab.Priors(intercept=ab.Normal(0, 2.5), b=ab.Normal(0, 1), sd=ab.
    HalfNormal(1)))
```

Fit 7.3 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 593. Predicted category probabilities at $x = -1, 0, 1$ (population level): (0.28, 0.27, 0.20, 0.14, 0.11); (0.17, 0.22, 0.22, 0.20, 0.19); (0.10, 0.16, 0.20, 0.23, 0.30).

parameter	true	posterior mean	90% interval
x	0.80	0.63	[0.54, 0.72]
Intercept[1]	-1.50	-1.56	[-1.85, -1.27]
Intercept[2]	-0.30	-0.42	[-0.71, -0.15]
Intercept[3]	0.60	0.48	[0.21, 0.77]
Intercept[4]	1.80	1.48	[1.20, 1.77]
1|g_sigma	0.70	0.76	[0.57, 1.00]

Zero-inflated Poisson counts

```
ab.zero_inflated_poisson("y ~ x + (1 | g)", df,
    ab.Priors(intercept=ab.Normal(0, 2.5), b=ab.Normal(0, 1), sd=ab.HalfNormal(1), zi=ab.Beta(2, 2)))
```

Fit 6.4 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 443.

parameter	true	posterior mean	90% interval
Intercept	0.60	0.66	[0.51, 0.82]
x	0.50	0.49	[0.45, 0.52]
zi	0.30	0.30	[0.28, 0.33]
1|g_sigma	0.50	0.51	[0.40, 0.63]

Distributional regression: the residual SD has its own formula

```
ab.gaussian(["y ~ x", "sigma ~ z"], df,
    ab.Priors(intercept=ab.Normal(0, 5), b=ab.Normal(0, 1), sigma={"intercept": ab.Normal(0, 1), "b": ab.
    Normal(0, 1)}))
```

Fit 1.6 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 4436.

parameter	true	posterior mean	90% interval
Intercept	1.00	0.99	[0.96, 1.02]
x	0.50	0.50	[0.47, 0.54]
sigma_Intercept	-0.20	-0.24	[-0.28, -0.20]
sigma_z	0.60	0.58	[0.54, 0.62]

Multivariate regression with a residual correlation

```
resp = {"intercept": ab.Normal(0, 5), "b": ab.Normal(0, 1), "sigma": ab.HalfNormal(1)}
ab.gaussian("mvbind(y1, y2) ~ x", df, ab.Priors(resp={"y1": resp, "y2": resp}, rescor=ab.LKJ(2)), rescor=True)
```

Fit 3.6 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 3132.

parameter	true	posterior mean	90% interval
y1_Intercept	–	0.49	[0.42, 0.56]
y1_x	1.00	1.02	[0.95, 1.09]
y2_Intercept	–	-0.32	[-0.39, -0.25]
y2_x	0.40	0.42	[0.36, 0.49]
y1_sigma	–	1.05	[1.00, 1.10]
y2_sigma	–	1.01	[0.96, 1.06]
rescor	0.60	0.62	[0.57, 0.66]

Non-linear growth curve with a varying amplitude

```
ab.gaussian(["y ~ a * exp(b * x)", "a ~ 1 + (1 | g)", "b ~ 1"], df,
    ab.Priors(nlpars={"a": {"b": ab.Normal(2, 1), "sd": ab.HalfNormal(1)}, "b": {"b": ab.Normal(0, 1)}},
              sigma=ab.HalfNormal(1)), nl=True)
```

Fit 3.0 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.010, min bulk ESS 407.

parameter	true	posterior mean	90% interval
a_Intercept	2.00	1.91	[1.74, 2.08]
b_Intercept	–	0.50	[0.49, 0.51]
a_1|g_sigma	–	0.34	[0.23, 0.50]
sigma	0.40	0.37	[0.36, 0.39]

Approximate Gaussian process (Hilbert-space basis, 20 functions)

```
ab.gaussian("y ~ gp(x, k = 20, c = 5/4)", df, ab.Priors(intercept=ab.Normal(0, 2), sigma=ab.HalfNormal(1),
    sdgp=ab.HalfNormal(1), lscale=ab.InverseGamma(3, 1)))
```

Fit 7.8 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 2056.

parameter	true	posterior mean	90% interval
Intercept	–	0.09	[-1.10, 1.30]
sigma	0.30	0.28	[0.26, 0.30]
sdgp_x	–	1.27	[0.73, 2.05]
lscale_x	–	0.25	[0.19, 0.32]

Autoregressive residuals within each of 20 series

```
ab.gaussian("y ~ x + ar(time = t, gr = g, p = 1)", df,
    ab.Priors(intercept=ab.Normal(0, 5), b=ab.Normal(0, 1), sigma=ab.HalfNormal(1), ar=ab.Normal(0, 0.5)))
```

Fit 1.7 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 4494.

parameter	true	posterior mean	90% interval
Intercept	–	1.04	[0.98, 1.10]
x	–	0.80	[0.78, 0.83]
ar[1]	0.60	0.54	[0.49, 0.59]
sigma	0.50	0.48	[0.46, 0.50]

Two-component Gaussian mixture

```
comp = lambda m0: ab.Priors(intercept=ab.Normal(m0, 2), sigma=ab.HalfNormal(2))
ab.mixture("y ~ 1", df, ab.Priors(components=[comp(-1), comp(1)], theta=ab.Dirichlet(2)),
           families=["gaussian", "gaussian"])
```

Fit 3.1 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.000, min bulk ESS 2425.

parameter	true	posterior mean	90% interval
Intercept[1]	-2.00	-1.92	[-2.00, -1.83]
Intercept[2]	1.50	1.47	[1.38, 1.56]
sigma1	0.70	0.69	[0.63, 0.76]
sigma2	1.00	1.03	[0.97, 1.10]
theta[1]	0.35	0.34	[0.31, 0.37]
theta[2]	–	0.66	[0.63, 0.69]

Measurement error in a predictor (the naive slope is attenuated)

```
ab.gaussian("y ~ me(xobs, sdx)", df, ab.Priors(intercept=ab.Normal(0, 5), b=ab.Normal(0, 2),
    sigma=ab.HalfNormal(1), meanme=ab.Normal(0, 2), sdme=ab.HalfNormal(1)))
```

Fit 3.7 s (four chains, 1000 + 1000 draws, NumPyro on the CPU); max $\hat{R}$ 1.010, min bulk ESS 505. The naive least-squares slope on the noisy predictor is 0.94 (attenuated); the true slope is 1.2.

parameter	true	posterior mean	90% interval
Intercept	–	0.52	[0.46, 0.58]
meanme_xobs	–	-0.07	[-0.15, 0.00]
me(xobs, sdx)	–	1.32	[1.24, 1.40]
sigma	–	0.46	[0.39, 0.53]
sdme_xobs	–	0.91	[0.85, 0.98]

Reading the tour. Every fit converged ($\hat{R} \leq 1.01$, smallest bulk ESS 407). Most true values lie inside their 90% intervals; three do not: the ordinal slope (0.63 against 0.8), the autoregressive coefficient (0.54 against 0.6, close to the known downward small-sample bias of AR estimates with 40-step series) and the measurement-error slope (1.32 against 1.2). I fitted these three data sets with brms as well: it gives 0.629, 0.539 and 1.323, the same posteriors, so the misses are properties of those simulated data sets, not of AutoBayes. These are also the first real fits of the multivariate, non-linear, Gaussian-process, autoregressive, mixture and measurement-error code paths; until now they had been checked against brms only through their densities, never sampled.

3 Head to head with brms

Set-up. Four models on the same simulated data (`examples/brms_parity/`), identical priors and sampler settings on both sides: four chains run in parallel, 1000 warm-up and 1000 kept draws each,

target acceptance 0.9. brms 2.23 with rstan 2.32 (Stan's NUTS, C++); AutoBayes with NumPyro (NUTS in JAX, compiled by XLA) on the CPU. brms compiles its Stan program first; AutoBayes's JIT compilation is part of its fit and is estimated separately with a one-draw fit. Effective sample sizes and $\hat{R}$ for both come from the same ArviZ estimators. The machine also ran its desktop applications (load average 15–30 during the runs); both libraries ran back to back under the same conditions, once each. brms used rstan (cmdstanr is not installed here).

Table 1: Wall-clock seconds, one run each. AutoBayes's total also includes building the model (0.1–11 s); its compile time is estimated by a one-draw fit.

		brms (rstan)			AutoBayes (NumPyro)			
model	rows	compile	sample	total	compile	sample	total	speed-up
B1 logistic slopes	2,000	37	6	43	4	1	8	5.3×
B2 Gaussian slopes	20,000	34	79	113	3	17	31	3.6×
B3 ordinal	1,500	28	34	62	3	5	7	8.5×
B4 zero-infl. Poisson	2,000	26	15	41	2	4	6	7.1×

Table 2: Sampling efficiency: the smallest bulk ESS over the compared parameters, per second of sampling and per second end to end; and the largest disagreement between the two posteriors, in combined Monte Carlo standard errors.

	brms			AutoBayes			
model	min ESS	per s sampling	per s total	min ESS	per s sampling	per s total	max $\lvert z \rvert$
B1 logistic slopes	1017	162	23.6	1264	1716	155.4	1.9
B2 Gaussian slopes	144	2	1.3	125	7	4.0	1.2
B3 ordinal	558	17	9.0	530	116	72.9	0.4
B4 zero-infl. Poisson	414	27	10.0	386	107	66.4	1.5

What the numbers say. End to end, AutoBayes was 3.6–8.5 times faster. For brms, 26–37 seconds of every fit is compiling the model's C++; brms can skip that when the same model is refitted (`update()` or a cached `file=`), so for repeated fits of one model the relevant comparison is the sampling column, where AutoBayes was also faster on all four models (roughly 4–9 times; its sampling time is estimated by subtracting a one-draw fit, so read it as approximate). Effective sample sizes per parameter are similar between the two, so the speed is not bought with worse mixing, and the posteriors agree (appendix). The largest model (B2, 20,000 rows) is the closest race (3.6 times end to end), partly because AutoBayes spends 11 seconds building that model through Bambi; its intercept mixes slowly in both libraries ($\hat{R}$ 1.02 for brms, 1.04 for AutoBayes), so its ESS figures are the least reliable.

What they do not say. These are four models on one laptop, one run each. I have not measured larger data, many more groups, or a GPU (where JAX may help further, but that is untested here), and brms with cmdstanr might sample somewhat faster than with rstan. The speed claim I would stand behind is: on typical hierarchical models of a few thousand rows, AutoBayes gets you a posterior that agrees with brms's several times sooner, mostly because there is no C++ compilation step.

4 Where AutoBayes is better than brms, and where it is not

Better, with evidence.

- *Turnaround.* No C++ compilation; formula to posterior in seconds (Section 3).
- *Numerical accuracy in the tails.* At frozen points, brms's complementary log-log gradient is 3.8×10^{-7} off the exact value, its censored-Gaussian gradient 2.5×10^{-6} off (Stan's `normal_lcdf` uses polynomial approximations), and its truncated-Gaussian expected value 10^{-3} off; its zero-inflated and hurdle Poisson gradients become NaN or infinite at extreme linear predictors, its cumulative probabilities lose digits between close thresholds, and loo's Monte Carlo error is NaN below an elpd of about -372. AutoBayes uses stable forms, each checked against 50-digit references (divergences D18, D24, D25, D28, D39, D50).
- *Same target, proven.* Every model class is tested deterministically against brms itself, coordinate by coordinate, on 98 frozen fixtures; unverified combinations are refused with a reason rather than silently built.
- *Python.* Results are ArviZ `InferenceData`, predictions are xarray, data are pandas; the sampler is JAX, which opens GPUs and vectorised chains (not yet validated).

Not better.

- *Breadth.* brms offers 49 families, many more terms and priors, and a large post-processing toolkit (`conditional_effects`, `hypothesis`, `pp_check`, `loo_compare`, emmeans); AutoBayes covers the core (Section 1).
- *Maturity.* brms has years of use and a community; AutoBayes's newest model classes were first sampled for this report, and none has had a recovery or calibration study beyond the tour.
- *Defaults.* AutoBayes asks for every prior explicitly (brms's defaults are available on request with `ab.default_priors()`); some will see that as better, others as friction.

5 What comes next

1. **Tier D (proposed), in the same oracle-gated loop.** The families researchers ask for next: Weibull, exponential, skew-normal, beta-binomial, zero-inflated and hurdle negative binomial and Gamma/lognormal hurdles, `sratio`/`cratio`/`acat`; post-processing: `loo_compare`, `waic`, `kfold`, `hypothesis`, `conditional_effects`; shrinkage priors (horseshoe, R2D2); `cs()`, `mm()`, `gr(by =)`. I would start by freezing the brms fixtures, as before.
2. **Validation beyond the oracle.** Simulation-based calibration or recovery studies for each family, and a first GPU run, both of which need your go-ahead (sampling budget, hardware).
3. **Open decisions.** The `s()` licensing question (clean-room rewrite recommended); GitHub Actions billing (the last nine merges went in on local verification, as you allowed).
4. **Small fixes found on the way.** PyMC's float32 $\log 2$ in half-Student-t and half-Cauchy priors (1.9×10^{-9} per prior); a per-row log density for mixtures, so that `log_lik` and `loo` work on them; `weights()` for zero-inflated and hurdle families.

A Posterior agreement, parameter by parameter

		posterior mean		posterior sd		
model	parameter	brms	AutoBayes	brms	AutoBayes	z
B1 logistic slopes	`b_Intercept`	-0.264	-0.273	0.113	0.111	-1.88
B1 logistic slopes	`b_x`	0.750	0.747	0.079	0.077	-1.32
B1 logistic slopes	`sd_g__Intercept`	0.689	0.687	0.095	0.092	-0.56
B1 logistic slopes	`sd_g__x`	0.357	0.361	0.090	0.089	1.17
B1 logistic slopes	`cor_g__Intercept__x`	0.397	0.395	0.222	0.212	-0.30
B2 Gaussian slopes	`b_Intercept`	1.971	1.967	0.069	0.074	-0.40
B2 Gaussian slopes	`b_x`	0.754	0.751	0.037	0.039	-1.22
B2 Gaussian slopes	`b_z`	-0.383	-0.383	0.011	0.011	-0.22
B2 Gaussian slopes	`sd_g__Intercept`	1.015	1.014	0.052	0.051	-0.34
B2 Gaussian slopes	`sd_g__x`	0.520	0.519	0.029	0.029	-0.17
B2 Gaussian slopes	`cor_g__Intercept__x`	-0.140	-0.135	0.071	0.071	1.08
B2 Gaussian slopes	`sigma`	1.500	1.500	0.008	0.007	-0.35
B3 ordinal	`b_Intercept[1]`	-1.379	-1.383	0.171	0.161	-0.34
B3 ordinal	`b_Intercept[2]`	-0.195	-0.197	0.166	0.157	-0.21
B3 ordinal	`b_Intercept[3]`	0.741	0.740	0.167	0.160	-0.13
B3 ordinal	`b_Intercept[4]`	1.962	1.962	0.175	0.166	-0.03
B3 ordinal	`b_x`	0.858	0.858	0.053	0.052	0.26
B3 ordinal	`sd_g__Intercept`	0.807	0.805	0.122	0.124	-0.41
B4 zero-infl. Poisson	`b_Intercept`	0.547	0.545	0.080	0.077	-0.41
B4 zero-infl. Poisson	`b_x`	0.501	0.501	0.020	0.020	0.96
B4 zero-infl. Poisson	`sd_g__Intercept`	0.464	0.469	0.059	0.060	1.46
B4 zero-infl. Poisson	`zi`	0.308	0.308	0.014	0.014	-0.39